

Phase 1: Imports

- 1.1. Import the core React library.
- 1.2. Import and destructure the three required named hooks—`createContext`, `useContext`, and `useReducer`—from the React package.

Phase 2: Create Context (The Intercom)

- 2.1. Call `createContext()` at the top-level module scope (outside all component functions).
- 2.2. Assign the result to a PascalCase variable name that represents your global channel.

Phase 3: Reducer Function (The Rulebook)

- 3.1. Declare a standalone function that accepts two arguments: `state` and `action`.
- 3.2. Set up a `switch` statement to evaluate `action.type`.
- 3.3. Write the "add" case to return a brand-new array containing all spread items from existing state plus the new item attached from the action payload.
- 3.4. Write the "clear" case to return an empty array.
- 3.5. Write a `default` case to return the current state unchanged as a safe fallback.

Phase 4: Main App Component (Provider & State Setup)

- 4.1. Declare and export your root `App` component function.
- 4.2. Initialize `useReducer` at the top of your component, passing your reducer function and an empty array as initial state to receive `cart` and `dispatch`.
- 4.3. Return your application layout wrapped inside the Context Provider component.
- 4.4. Pass an object containing both `cart` and `dispatch` into the Provider's `value` prop.
- 4.5. Include your structural elements and child components inside the Provider.

Phase 5: Product List Component (Display Products)

- 5.1. Create an array of mock product strings or objects.
- 5.2. Loop over the product array using the `.map()` method to generate a row for each item.
- 5.3. Assign a unique `key` prop to the outermost JSX wrapper of each mapped item.
- 5.4. Render the item text alongside a child button component, passing the current product down as a prop.

Phase 6: Add to Cart Button Component (Dispatches 'add')

- 6.1. Destructure the product item prop inside the component arguments.
- 6.2. Access the global `dispatch` function by calling `useContext` with your Context channel.

- **6.3.** Attach an `onClick` event handler to the button that triggers `dispatch` with an action object containing the `"add"` type and the item payload.

Phase 7: Cart Display Component (Reads Cart & Dispatches 'clear')

- **7.1.** Call `useContext` with your Context channel to retrieve both `cart` and `dispatch`.
- **7.2.** Set up a conditional branch to check if the cart array is empty.
- **7.3.** Render an empty cart message if length is 0, or map over the `cart` array to render each added item in a list if items exist.
- **7.4.** Add a clear button with an `onClick` event handler that calls `dispatch` with an action object containing the `"clear"` type.